Friday Assistant

Generated: 2026-05-24 11:00

Table of Contents

Com

Files ir

FILE: build_exe.bat

```
@echo off
REM Bui
setlocal
cd /d "%
if not exist .venv (
echo Vir
call .venv\Scripts\activate.bat
python -
endlocal
```

-- end of build_exe.bat (16 lines) --

FILE: Enable Friday at Login.bat

```
@echo off
REM Star


set STARTUP=%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup
set LINK

powershell -NoProfile -Command ^
"$s = Ne




echo.
echo Don


```

-- end of Enable Friday at Login.bat (23 lines) --

FILE: friday__init__.py

```
"""Friday - personal AI assistant package."""

__version__ = "0.1.0"
                                                                                    __app_na
```

-- end of friday__init__.py (5 lines) --

FILE: friday__main__.py

```
"""Allow ``python -m friday`` and ``python -m friday.background_listener``."""

import sys

if __name__ == "__main__":
                                                                        if len(s

raise SystemExit(main())
                                                                        from fri

raise SystemExit(main())
```

-- end of friday__main__.py (13 lines) --

FILE: friday\background_listener.py

```
"""Always-on background wake-word listener.

Runs in the background (use ``pythonw -m friday.background_listener``) and
listens

Say "Friday" alone to open the assistant, or "Friday, open Chrome" to open
and run

from __future__ import annotations

import logging
import s

from friday.config import load_settings
from fri




log = logging.getLogger(__name__)

# Avoid launching twice in quick succession.
_DEBOUNC

def run_listener() -> int:
settings




stt = STTEngine()
if not s




ipc.register_listener()
wake = s



last_activation = 0.0

try:
while Tr









if not text:
continue

if wake not in text.lower():
continue

now = time.monotonic()
if now -
```

```
command = detector.extract_command_after_wake(text)
                                                                                                                    if comma



activate_friday(pending_command=command)
                                                                                                                    time.sle




return 0

def main() -> int:
                                                                                                                    setup_lo


if __name__ == "__main__":
                                                                                                                    sys.exit
```

-- end of friday\background_listener.py (93 lines) --

FILE: friday\config.py

```
"""Persistent user configuration for Friday.

Settings live at ``%USERPROFILE%/.friday/config.json``. The file is created
with ser

Friday never stores or reads anything from outside its own config directory -
this mod


from __future__ import annotations

import json
import 




log = logging.getLogger(__name__)

# Friday keeps its own private folder under the user's home directory.
# This 




DEFAULT_APP_ALLOWLIST: Dict[str, str] = {
# friend

}

DEFAULT_SITE_SHORTCUTS: Dict[str, str] = {
"youtube

}


@dataclass
class Vo
```

```
@dataclass
class Br

@dataclass
class We

@dataclass
class Po

@dataclass
class UI

@dataclass
class Ap

def to_dict(self) -> Dict[str, Any]:
return a

@classmethod
def from
```

```
def ensure_app_dir() -> None:
                                                                                                                    APP_DIR.
```

```
def load_settings() -> AppSettings:
                                                                                                                    ensure_a
```

```
def save_settings(settings: AppSettings) -> None:
                                                                                                                    ensure_a
```

-- end of friday\config.py (174 lines) --

FILE: friday\core__init__.py

```
"""Core command-routing brain for Friday."""

from friday.core.router import CommandRouter, RouterResult

__all__ = ["CommandRouter", "RouterResult"]
```

-- end of friday\core__init__.py (6 lines) --

FILE: friday\core\brain.py

```
"""Conversational fallbacks and small-talk replies."""

from __future__ import annotations

import datetime as _dt
import r

_GREETING_REPLIES: List[str] = [
"Online

_THANK_REPLIES: List[str] = [
"Always

_JOKES: List[str] = [
"I told

_SMALLTALK_REPLIES: List[str] = [
"Interes

_HELP_TEXT = (
"I can c

def greet(user_name: str) -> str:
return r

def thank(user_name: str) -> str:
return r

def joke() -> str:
return r

def smalltalk(user_name: str) -> str:
return r

def help_text(user_name: str) -> str:
```

```
                                                                                return _

def now_time() -> str:
                                                                                return _

def now_date() -> str:
                                                                                return _

def boot_greeting(user_name: str) -> str:
                                                                                hour = _
```

-- end of friday\core\brain.py (91 lines) --

FILE: friday\core\intents.py

```
"""Intent classification.

A very lightweight rules + fuzzy-match parser. We deliberately avoid an LLM
                                                                                depender



from __future__ import annotations

import re
                                                                                from dat



class IntentName(Enum):
                                                                                OPEN_APP









@dataclass
                                                                                class In



_GREET_PATTERNS = (
                                                                                "hello",









_THANK_PATTERNS = ("thank you", "thanks", "thank u", "much appreciated")

_EXIT_PATTERNS = (
                                                                                "exit",
```

```
_JOKE_PATTERNS = ("tell me a joke", "make me laugh", "joke please", "say something funny")
_HELP_PATTERNS = ("help", "what can you do", "what can you do for me", "commands", "how do you work")

def _strip_wake_word(text: str, wake_word: str) -> str:
                                                                                                                    cleaned


def classify(text: str, wake_word: str = "friday") -> Intent:
                                                                                                                    raw = te


lowered = _strip_wake_word(raw.lower(), wake_word.lower())
if any(p in lowered for p in _EXIT_PATTERNS):
                                                                                                                    return I


if "time" in lowered and ("what" in lowered or "tell" in lowered or "current" in lowered):
                                                                                                                    return I


weather_match = re.search(
                                                                                                                    r"weathe


if any(p in lowered for p in ("news", "headlines", "what's happening", "whats happening", "world today", "going on in the
world")):
                                                                                                                    topic_ma


open_match = re.match(
                                                                                                                    r"^\s*(?
```

```
if any(lowered.startswith(p) for p in ("go to ", "navigate to ", "browse to ")):
                                                                                                                target =

# Search verbs.
                                                                                                                search_m









# As a last resort treat any non-trivial phrase as smalltalk.
                                                                                                                if len(
```

-- end of friday\core\intents.py (152 lines) --

FILE: friday\core\router.py

```
"""Command router - translates parsed intents into actions.

Every command, whether it came from text or voice, ends up here. The router
is respo

1. Running the input through the safety policy.
2. Class



The router never touches the user's files. If a future intent ever needs
to, it m



from __future__ import annotations

import logging
from dat

from friday.config import AppSettings
from fri









log = logging.getLogger(__name__)

@dataclass
class Ro









class CommandRouter:
    def __in

    # ------------------------------------------------------------------
    # Public



    decision = evaluate(command)
    if decis
```

```
intent = classify(command, wake_word=self.settings.voice.wake_word)
                                                                                                log.info
try:
                                                                                                return s
# ----------------------------------------------------------------
                                                                                                # Dispat
if name is IntentName.GREET:
                                                                                                return F
if name is IntentName.OPEN_APP:
                                                                                                return s
return RouterResult(
                                                                                                spoken=(
# ----------------------------------------------------------------
```

```
# Specif

# Browsers map to the browser integration so we honour the preferred
# browse

ok, msg = apps_integration.launch_app(self.settings, target)
return R

def _open_site(self, target: str) -> RouterResult:
target =

def _web_search(self, query: str) -> RouterResult:
query =

def _news(self, topic: str) -> RouterResult:
result =

def _weather(self, city: str) -> RouterResult:
result =
```

-- end of friday\core\router.py (185 lines) --

FILE: friday\installer__init__.py

```
"""Friday installer / packaging helpers."""
```

-- end of friday\installer__init__.py (2 lines) --

FILE: friday\installer\build_installer.py

```
"""Build a Windows-installable Friday using PyInstaller.

Run from the project root:

python -m friday.installer.build_installer

The result is placed under ``dist/Friday/`` - copy this folder anywhere on
your PC

For a guided MSI installer, install ``inno-setup`` separately and feed it
``dist/F

from __future__ import annotations

import logging
import s

log = logging.getLogger(__name__)

def _project_root() -> Path:
return P

def main() -> int:
logging

if not spec.exists():
log.erro

if shutil.which("pyinstaller") is None:
log.erro

cmd = [
"pyinsta

out_dir = root / "dist" / "Friday"
log.info

if __name__ == "__main__":
```

```
sys.exit
```

-- end of friday\installer\build_installer.py (69 lines) --

FILE: friday\installer\Friday.spec

```
# PyInstaller spec for Friday - produces a one-folder Windows app.
#

from pathlib import Path

import PyInstaller.utils.hooks as hooks

block_cipher = None
project_

hidden = []
hidden -

datas = []
assets_c

a = Analysis(
[entry],

pyz = PYZ(a.pure, a.zipped_data, cipher=block_cipher)

icon_path = project_root / "assets" / "icon.ico"
exe = E
```

```
coll = COLLECT(
                                                                                                                exe,
```

-- end of friday\installer\Friday.spec (90 lines) --

FILE: friday\integrations__init__.py

```
"""External integrations: browser, apps, search, news, weather."""
```

-- end of friday\integrations__init__.py (2 lines) --

FILE: friday\integrations\apps.py

```
"""Application launching adapter.

Friday only launches programs that are present in the configured allowlist.
The allo

We deliberately do NOT call ``os.startfile`` on arbitrary paths from the
user's m



from __future__ import annotations

import logging
import s



from rapidfuzz import process, fuzz

from friday.config import AppSettings

log = logging.getLogger(__name__)

def _resolve_target(settings: AppSettings, target: str) -> str | None:
    """Find



    allowlist = settings.app_allowlist
    if targe

    candidates = list(allowlist.keys())
    if not c









def launch_app(settings: AppSettings, target: str) -> Tuple[bool, str]:
    command



    if not _is_on_path(command):
        return (
```

```
try:
# `start

return True, f"Opening {target}."

def _is_on_path(executable: str) -> bool:
if not e
```

-- end of friday\integrations\apps.py (95 lines) --

FILE: friday\integrations\browser.py

```
"""Browser controller - open Chrome / Brave / default browser, open URLs.

Friday refuses to navigate to anything that looks like a file URL. URL
                                                                                sanitisa


from __future__ import annotations

import logging
                                                                                import s




from friday.config import AppSettings

log = logging.getLogger(__name__)

# Common Windows install locations for popular browsers. Used only as a
                                                                                # fallba
```

```
def _expand(path: str) -> str:
                                                                                import o


def _find_browser_executable(name: str) -> str | None:
                                                                                name = n
```

```
def _looks_like_safe_url(url: str) -> bool:
                                                                                """Rejec
```

```
def _normalise_target(settings: AppSettings, target: str) -> str | None:
                                                                                target =
```

```
lower = target.lower()
if lower
if "://" in target:
return t
# Looks like a domain (has a dot) - prepend https.
if "." i
# Otherwise treat as a search query.
return t
def open_browser(settings: AppSettings, browser: str | None = None) -> Tuple[bool, str]:
"""Open
try:
subproce
def open_url(settings: AppSettings, target: str) -> Tuple[bool, str]:
url = _n
chosen = (settings.browser.preferred or "chrome").lower()
exe = _f
try:
if exe i
```

-- end of friday\integrations\browser.py (122 lines) --

FILE: friday\integrations\news.py

```
"""News headlines adapter.

Two providers:
* NewsAP

Returns a list of human-readable headlines plus the source URLs.
"""

from __future__ import annotations

import logging
from dat

import feedparser
import r

from friday.config import AppSettings

log = logging.getLogger(__name__)

@dataclass
class Ne


_NEWS_API_ENDPOINT = "https://newsapi.org/v2/top-headlines"
_USER_AG

# Reliable RSS feeds for the no-key fallback.
_RSS_FEE



def _format_headlines(headlines: List[tuple[str, str]]) -> str:
    if not h



def _newsapi_headlines(api_key: str, topic: str) -> NewsResult:
    params =
```

```
articles = data.get("articles") or []
headline

def _rss_headlines(topic: str) -> NewsResult:
headline

for source_name, url in _RSS_FEEDS:
try:

if not headlines and topic_lc:
# If fil

return NewsResult(_format_headlines(headlines), success=bool(headlines), sources=sources)

def headlines(settings: AppSettings, topic: str = "") -> NewsResult:
if sett
```

-- end of friday\integrations\news.py (123 lines) --

FILE: friday\integrations\weather.py

```
"""Weather adapter.

Default provider: Open-Meteo (free, no API key, https://open-meteo.com).
Optional

from __future__ import annotations

import logging
from dat

import requests

from friday.config import AppSettings

log = logging.getLogger(__name__)

@dataclass
class We

_GEOCODE_URL = "https://geocoding-api.open-meteo.com/v1/search"
_OPEN_ME

_WEATHER_CODES = {
0: "clea

def _geocode(city: str) -> tuple[float, float, str] | None:
try:
```

```
def _open_meteo(city: str) -> WeatherResult:
                                                                                                    geo = _g
cur = data.get("current") or {}
                                                                                                    temp = c
if temp is None:
                                                                                                    return W
spoken = (
                                                                                                    f"In {la
def _openweather(city: str, api_key: str) -> WeatherResult:
                                                                                                    try:
```

```
main = data.get("main") or {}
                                                                         weather

def current(settings: AppSettings, city: str = "") -> WeatherResult:
                                                                         chosen =

if settings.web.weather_api_key:
                                                                         return _
```

-- end of friday\integrations\weather.py (162 lines) --

FILE: friday\integrations\web_search.py

```
"""Web search adapter.

Default provider is DuckDuckGo's Instant Answer API which requires no key.
Optional

Returns a structured :class:`SearchResult` so the router can both speak the
top answ


from __future__ import annotations

import logging

from dat


import requests

from friday.config import AppSettings

log = logging.getLogger(__name__)


@dataclass

class Se




_DDG_ENDPOINT = "https://api.duckduckgo.com/"

_SERPAPI



def _ddg_search(query: str) -> SearchResult:

    try:

        abstract = (data.get("AbstractText") or "").strip()

        answer =

        sources: List[str] = []

        if abstr
```

```
spoken_bits: List[str] = []
if headi

if not spoken_bits:
# Fall b

return SearchResult(spoken=" ".join(spoken_bits), success=True, sources=sources)

def _serpapi_search(query: str, api_key: str) -> SearchResult:
try:

answer_box = data.get("answer_box") or {}
snippet

organic = data.get("organic_results") or []
if not s

if not snippet:
return S
```

```
def search(settings: AppSettings, query: str) -> SearchResult:
                                                                                                                                                query =

provider = (settings.web.search_provider or "duckduckgo").lower()
                                                                                                                                                if provi
```

-- end of friday\integrations\web_search.py (145 lines) --

FILE: friday\ipc.py

```
"""Inter-process communication for Friday.

The background wake listener and the main GUI communicate through small
files u
* ``gui.lock`` - PID of the running GUI (if any)
* ``list



from __future__ import annotations

import logging
import c



from friday.config import APP_DIR, ensure_app_dir

log = logging.getLogger(__name__)

GUI_LOCK = APP_DIR / "gui.lock"
LISTENER



def _pid_alive(pid: int) -> bool:
    if pid <



PROCESS_QUERY_LIMITED_INFORMATION = 0x1000
    handle =



def _read_pid(path: Path) -> Optional[int]:
    if not p
```

```
def _write_pid(path: Path, pid: int) -> None:
ensure_a

def register_gui(pid: int | None = None) -> None:
_write_p

def unregister_gui() -> None:
try:

def is_gui_running() -> bool:
pid = _r

def register_listener(pid: int | None = None) -> None:
_write_p

def unregister_listener() -> None:
try:

def is_listener_running() -> bool:
pid = _r

def queue_command(command: str) -> None:
ensure_a

def pop_pending_command() -> Optional[str]:
if not P
```

```
def signal_wake() -> None:
                                                                                                                                ensure_a


def consume_wake_signal() -> bool:
                                                                                                                                if not W
```

-- end of friday\ipc.py (157 lines) --

FILE: friday\launcher.py

```
"""Launch or focus the Friday GUI from the background listener."""

from __future__ import annotations

import logging
import c




from friday import ipc

log = logging.getLogger(__name__)

def project_root() -> Path:
    return P

def python_executable() -> str:
    """Prefe




def pythonw_executable() -> str:
    venv_pyw




def focus_friday_window() -> bool:
    """Bring









found = False

def _callback(hwnd, _extra) -> None:
    nonloca









try:
    win32gui
```

```
def activate_friday(pending_command: str | None = None) -> None:
                                                                """Open

    if ipc.is_gui_running():
                                                                    ipc.sign
        ipc.signal_wake()
                                                                    exe = py

    try:
                                                                    subproce


def start_background_listener() -> bool:
                                                                """Start

    exe = pythonw_executable()
                                                                    root = p

    try:
                                                                    subproce
```

-- end of friday\launcher.py (123 lines) --

FILE: friday\logging_setup.py

```
"""Logging configuration for Friday."""

from __future__ import annotations

import logging
import s


from friday.config import LOG_DIR, ensure_app_dir

def setup_logging(level: int = logging.INFO) -> None:
    ensure_a


    formatter = logging.Formatter(
        fmt="%(a


    file_handler = RotatingFileHandler(
        log_file


    stream_handler = logging.StreamHandler(sys.stdout)
    stream_h

    root = logging.getLogger()
    root.set
```

-- end of friday\logging_setup.py (34 lines) --

FILE: friday\main.py

```
"""Friday entry point.

Boots logging, loads settings, builds the router + UI + voice engine, and
runs the

from __future__ import annotations

import logging
import s

from PySide6.QtCore import QTimer
from PyS

from friday import __app_name__
from fri

log = logging.getLogger(__name__)

def _check_internet() -> bool:
    """Quick

def _wire_voice(window: MainWindow, voice: Optional[VoiceEngine]) -> None:
    if voice

    def on_state(state: VoiceState) -> None:
        if state

    voice.state_changed.connect(on_state)
    voice.st

    window.speak_requested.connect(voice.speak)
    window.v

def _ensure_background_listener(settings) -> None:
    if setti
```

```
def _wire_ipc(window: MainWindow) -> None:
    """Poll
    def _poll() -> None:
        if ipc.c









timer = QTimer()
timer.ti


def main() -> int:
    setup_lo

    settings = load_settings()
    save_set

    app = QApplication.instance() or QApplication(sys.argv)
    app.setA

    router = CommandRouter(settings)
    window =

    voice: Optional[VoiceEngine] = None
    try:




if voice is not None:
    _wire_vo









if settings.ui.always_on_top:
    from PyS

ipc.register_gui()
    _wire_ip
if settings.ui.start_minimized:
    window.s


# Command queued before the window finished opening.
pending
```

```
try:
                                                                                                                exit_cod
return exit_code

if __name__ == "__main__":
                                                                                                                raise Sy
```

-- end of friday\main.py (152 lines) --

FILE: friday\policy__init__.py

```
"""Friday safety policy package.

The policy engine guarantees Friday will never read, write, or even browse
                                                                                                                                         the user

from friday.policy.guardrails import (
                                                                                                                                         PolicyDe

__all__ = ["PolicyDecision", "PolicyVerdict", "evaluate"]
```

-- end of friday\policy__init__.py (15 lines) --

FILE: friday\policy\guardrails.py

```
"""Safety guardrails for Friday.

Friday is allowed to:
                                                                                * Open C




Friday is NEVER allowed to:
                                                                                * Read,




This file is the single source of truth for what is denied. The router will
                                                                                NOT disp


from __future__ import annotations

import logging
                                                                                import r




log = logging.getLogger(__name__)

class PolicyVerdict(Enum):
                                                                                ALLOW =


@dataclass
                                                                                class Po




@property
                                                                                def allo


# ---------------------------------------------------------------------------
                                                                                # Block


_FILE_VERBS = (
                                                                                "read",
```

```
_FILE_NOUNS = (
                                                                                                                                "file",
```

```
# Phrases that are always blocked, no matter what verbs surround them.
_HARD_BL

# Compiled once for speed.
_PATH_RE

# Word-boundary hard-block tokens. These catch short system tokens like
# "cmd"
```

```
def _contains_phrase(haystack: str, phrases: Iterable[str]) -> Tuple[bool, str]:
for phra

def _looks_like_file_action(lowered: str) -> Tuple[bool, str]:
"""Detec

def evaluate(command: str) -> PolicyDecision:
"""Evalu
Returns a :class:`PolicyDecision`. Callers MUST refuse to run anything
whose ve

text = command.strip()
if not t
lowered = text.lower()
blocked, phrase = _contains_phrase(lowered, _HARD_BLOCK_PHRASES)
if block

for regex in _HARD_BLOCK_REGEXES:
m = rege
```

```
if _PATH_REGEX.search(text):
                                                                                                                log.info
```

```
is_file_action, why = _looks_like_file_action(lowered)
                                                                                                                if is_fi
```

```
return PolicyDecision(PolicyVerdict.ALLOW, "", "ok")
```

-- end of friday\policy\guardrails.py (281 lines) --

FILE: friday\ui__init__.py

```
"""Friday futuristic UI package (PySide6)."""

from friday.ui.main_window import MainWindow

__all__ = ["MainWindow"]
```

-- end of friday\ui__init__.py (6 lines) --

FILE: friday\ui\chat_view.py

```
"""Chat transcript widget with neon bubbles."""

from __future__ import annotations

from typing import Iterable, List

from PySide6.QtCore import Qt, QTimer
```

```
from PyS
```

```
class _Bubble(QFrame):
```

```
def __in
```

```
layout = QVBoxLayout(self)
```

```
layout.s
```

```
speaker_label = QLabel(speaker.upper())
```

```
speaker_
```

```
body = QLabel(text)
```

```
body.set
```

```
if sources:
```

```
for src
```

```
self.setSizePolicy(QSizePolicy.Maximum, QSizePolicy.Preferred)

class ChatView(QScrollArea):
```

```
"""Verti
```

```
def __init__(self, parent: QWidget | None = None):
```

```
super()
```

```
self._content = QWidget()
```

```
# ------------------------------------------------------------------
```

```
wrapper = QWidget()
```

```
def _scroll_to_bottom(self) -> None:
```

```
# ------------------------------------------------------------------
```

```
def add_assistant(self, text: str, sources: List[str] | None = None) -> None:
```

```
def add_blocked(self, text: str) -> None:
```

```
def add_system(self, text: str) -> None:
```

-- end of friday\ui\chat_view.py (110 lines) --

FILE: friday\ui\command_input.py

```
"""Bottom command input row: text field + mic button."""

from __future__ import annotations

from PySide6.QtCore import Qt, Signal
from PyS

class CommandInput(QWidget):
submitte

def __init__(self, parent: QWidget | None = None):
super().

self._input = QLineEdit()
self._in

self._mic = QPushButton("MIC")
self._mi

send = QPushButton("SEND")
send.set

layout.addWidget(self._input, 1)
layout.a

def _on_return(self) -> None:
text = s

# ------------------------------------------------------------------
def set_

def focus_input(self) -> None:
self._in
```

-- end of friday\ui\command_input.py (55 lines) --

FILE: friday\ui\main_window.py

```
"""Friday's main window - the visible UI shell.

Bridges the UI to the command router and the voice engine. The window is
deliberately thin: it owns widgets and forwards events, nothing more.
"""

from __future__ import annotations

import logging

from typing import TYPE_CHECKING

from PySide6.QtCore import Qt, QThread, Signal

if TYPE_CHECKING:
    from friday.config import AppSettings
    from friday.core.router import CommandRouter, RouterResult

from friday import __app_name__

if TYPE_CHECKING:
    from PySide6.QtGui import QCloseEvent

log = logging.getLogger(__name__)


class _RouterWorker(QThread):
    """Run the router off the UI thread so the window never freezes."""

    result_ready = Signal(object)  # RouterResult

    def __init__(self, router: CommandRouter, command: str):
        super().__init__()
        self._router = router
        self._command = command

    def run(self) -> None:
        try:
            result = self._router.route(self._command)
        except Exception:
            log.exception("Router crashed on command: %r", self._command)
            return
        self.result_ready.emit(result)


class MainWindow(QMainWindow):
    """Friday's main window."""

    def __init__(self, settings: AppSettings, router: CommandRouter):
        super().__init__()
        self._settings = settings
        self._router = router

        self.setWindowTitle(f"{__app_name__} · Personal AI")
```

```
self.setStyleSheet(GLOBAL_QSS)
root = QWidget()
outer = QHBoxLayout(root)
self._hud = StatusHUD()
# Right-hand main panel.
chat_panel = QFrame()
self._input = CommandInput()
self._input.submitted.connect(self._handle_command)
# Boot greeting in the chat.
self._center_on_screen()
# ------------------------------------------------------------------
# ------------------------------------------------------------------
def display_assistant_text(self, text: str, sources: Optional[list] = None, blocked: bool = False) -> None:
```

```
self.res
root.set
outer.se
outer.ad
right =
chat_par
right_la
self._in
greeting
self._in
def _cen
# Public
if block
```

```
def set_voice_listening(self, listening: bool, label: str = "") -> None:
                                                                                                                    self._in
```

```
def set_voice_status(self, text: str) -> None:
                                                                                                                    self._hu
def set_internet_online(self, online: bool) -> None:
                                                                                                                    self._hu
def handle_voice_command(self, text: str) -> None:
                                                                                                                    if not t
```

```
# ------------------------------------------------------------------
                                                                                                                    def _har
```

```
def _dispatch(self, text: str) -> None:
                                                                                                                    self._hu
```

```
def _on_result(self, result: RouterResult) -> None:
                                                                                                                    self.dis
```

-- end of friday\ui\main_window.py (183 lines) --

FILE: friday\ui\status_hud.py

```
"""Sidebar HUD: brand, system clock, voice status indicator."""
from __future__ import annotations
import datetime as _dt
from PySide6.QtCore import Qt, QTimer
from PyS

class StatusHUD(QFrame):
def __in


layout = QVBoxLayout(self)
layout.s

title = QLabel("FRIDAY")
title.se

subtitle = QLabel("PERSONAL AI / v0.1")
subtitle

layout.addSpacing(12)
self._clock_value = QLabel("--:--:--")
self._cl


self._date_value = QLabel("")
self._da

layout.addSpacing(20)
layout.addWidget(self._make_label("STATUS", object_name="statusDetail"))
self._st

layout.addSpacing(12)
layout.a


layout.addSpacing(12)
layout.a


layout.addStretch(1)
# Live clock.
timer =


@staticmethod
```

```
def _tick(self) -> None:

# ---------------------------------------------------------------

def set_voice(self, text: str) -> None:

def set_internet(self, online: bool) -> None:
```

-- end of friday\ui\status_hud.py (88 lines) --

FILE: friday\ui\styles.py

```
"""Centralised QSS for Friday's neon HUD theme."""

NEON_PRIMARY = "#00E5FF" # cyan
NEON_ACC

GLOBAL_QSS = f"""
* {{

QMainWindow, QWidget#root {{
backgrou

QWidget#sidebar {{
backgrou

QLabel#brandTitle {{
font-siz

QLabel#brandSubtitle {{
font-siz

QLabel#statusLabel {{
font-siz

QLabel#statusDetail {{
font-siz

QFrame#chatPanel {{
backgrou

QScrollArea {{
backgrou

QWidget#chatContent {{
backgrou
```

```
QFrame#userBubble {{
                                                                                    backgrou

QFrame#assistantBubble {{
                                                                                    backgrou

QFrame#blockedBubble {{
                                                                                    backgrou

QLabel.bubbleText {{
                                                                                    font-siz

QLabel.bubbleSpeaker {{
                                                                                    font-siz

QLineEdit#commandInput {{
                                                                                    backgrou

QLineEdit#commandInput:focus {{
                                                                                    border:

QPushButton {{
                                                                                    backgrou

QPushButton:hover {{
                                                                                    backgrou

QPushButton:pressed {{
                                                                                    backgrou

QPushButton#micButton[listening="true"] {{
                                                                                    border:
```

```
SOURCE_LINK_QSS = f"""
                                                                                                    QLabel.s
```

-- end of friday\ui\styles.py (147 lines) --

FILE: friday\voice__init__.py

```
"""Voice subsystem: wake word, speech-to-text, text-to-speech.

All components are optional. If their dependencies are not installed at
                                                                        runtime,



from friday.voice.engine import VoiceEngine, VoiceState

__all__ = ["VoiceEngine", "VoiceState"]
```

-- end of friday\voice__init__.py (11 lines) --

FILE: friday\voice\engine.py

```
"""Voice engine - wires wake-word + STT + TTS together for the UI.

Runs the listening loop on a background thread and emits Qt signals that
the mai

from __future__ import annotations

import logging
from en

from PySide6.QtCore import QThread, Signal

from friday.config import AppSettings
from fr

log = logging.getLogger(__name__)

class VoiceState(Enum):
DISABLE

class VoiceEngine(QThread):
state_c

def __init__(self, settings: AppSettings):
super()

# ----------------------------------------------------------------
@proper

@property
def tts_

def is_running(self) -> bool:
return

def speak(self, text: str) -> None:
if not
```

```
def toggle(self) -> None:
                                                                """Pause
```

```
def stop(self) -> None:
                                                                self._st
```

```
# ------------------------------------------------------------------
                                                                def run(
```

```
self.status_message.emit(
                                                                f"Listen
```

```
while not self._stop.is_set():
                                                                if not s
```

```
self.state_changed.emit(VoiceState.AWAITING_WAKE)
                                                                try:
```

```
if not text:
                                                                continue
wake = self._settings.voice.wake_word.lower()
                                                                if wake
# The user already included a command in the same utterance?
                                                                tail = s
```

```
# Otherwise capture the follow-up command.
                                                                self.sta
```

```
if command:
                                                                                    self.com
self.state_changed.emit(VoiceState.IDLE)
```

-- end of friday\voice\engine.py (142 lines) --

FILE: friday\voice\stt.py

```
"""Speech-to-text helper using SpeechRecognition + Google's free recognizer.

The STT engine is intentionally lightweight: it captures one utterance after
being tr

If ``speech_recognition`` or PyAudio is missing, ``listen_once`` returns
``None``


from __future__ import annotations

import logging
from typ

log = logging.getLogger(__name__)

class STTEngine:
    def __in









def _init(self) -> None:
    try:
```

```
@property
    def avai

def listen_once(self, timeout: float = 6.0, phrase_time_limit: float = 8.0) -> Optional[str]:
    """Captu

Returns ``None`` if STT is unavailable or recognition fails.
    """


try:
    with se
```

```
try:
                                                                                                                    text = s
```

-- end of friday\voice\stt.py (80 lines) --

FILE: friday\voice\tts.py

```
"""Text-to-speech adapter.

Tries to use ``pyttsx3`` (offline, fast, robust on Windows). If pyttsx3 is
not inst


from __future__ import annotations

import logging
import t


log = logging.getLogger(__name__)

class TTSEngine:
    def __in


    def _init_engine(self) -> None:
        try:


        try:
            engine =


    @property
    def avai

    def speak(self, text: str) -> None:
        if not t

        def _run() -> None:
            with sel
```

```
thread = threading.Thread(target=_run, daemon=True)
                                                                                                                                                thread.s
def stop(self) -> None:
                                                                                                                                                if self.
```

-- end of friday\voice\tts.py (76 lines) --

FILE: friday\voice\wake_word.py

```
"""Wake-word detection.

The default implementation simply listens for a short utterance and checks
                                                                                whether


The detector exposes a single ``detect`` blocking call so the engine can
                                                                                loop on


from __future__ import annotations

import logging
                                                                                from typ

log = logging.getLogger(__name__)

class WakeWordDetector:
                                                                                def __in


def detect(self, stt_engine) -> bool:
                                                                                """Block

Uses the supplied STT engine to capture a short phrase and checks for
                                                                                the wake









def extract_command_after_wake(self, text: str) -> Optional[str]:
                                                                                """If ``
```

-- end of friday\voice\wake_word.py (54 lines) --

FILE: Generate PDFs.bat

```
@echo off
cd /d "%
```

-- end of Generate PDFs.bat (15 lines) --

FILE: install.bat

```
@echo off
REM Fri

setlocal
cd /d "%

set PY_LAUNCHER=
where py

if not exist .venv (
echo Cre

echo Installing dependencies...
call .ve

echo.
python
Python 3

endlocal
```

-- end of install.bat (55 lines) --

FILE: Launch Friday.bat

```
@echo off
title Fr
```

-- end of Launch Friday.bat (11 lines) --

FILE: pyproject.toml

```
[build-system]
requires

[project]
name = "

[project.scripts]
friday =

[tool.setuptools.packages.find]
include
```

-- end of pyproject.toml (20 lines) --

FILE: README.md

Friday - Personal AI Assistant

Friday is a JARVIS/F.R.I.D.A.Y.-inspired Windows desktop assistant with a

futurist

It is built with strong safety guardrails: **Friday will never read, open, or

modify y

Features

- Futuristic chat-style UI with neon HUD (PySide6)

- Voice

Quick start (development)

```powershell
# 1. clo

# 2. create a virtual environment

python -

# 3. install dependencies

pip inst

# 4. run Friday

python -
```

The first launch creates `~/.friday/config.json` with your default settings.

> Voice and wake-word features require a working microphone. If voice

> depend

Building an installable Windows executable

```powershell
python -
```

This produces `dist/Friday/Friday.exe` plus a portable folder you can copy

anywhere

Configuration

User-editable settings live at `%USERPROFILE%\.friday\config.json`.

Important keys:

| key | purpose |

| -----

Safety model

Friday operates in **trusted mode** by default. Approved actions run instantly.

However

- Any command referencing a file path, drive letter, `file://` URL, or

Explorer

These rules are enforced **before** any intent is dispatched. There is no

override

Project layout

```

friday-a

---

## Voice commands you can say

- "Friday, what's the weather in Mumbai?"

- "Frida

You can also type any of these into the command input at the bottom of the UI.

---

## Documentation (PDF)

Full guides for users and GitHub upload are in **[docs/pdf/](docs/pdf/)**:

| PDF | Contents |

|-----|-
```

Regenerate PDFs anytime:

```powershell
.\Genera
```

Markdown sources live in `docs/`.

Upload to GitHub

See docs/05_GitHub_Upload_Guide.md or the PDF version in `docs/pdf/`.

Quick steps:

```powershell
git init
```

License

MIT License - see LICENSE.

-- end of README.md (173 lines) --

FILE: requirements.txt

```
# Core UI
PySide6>

# HTTP + parsing for web/news/weather
requests



# Text-to-speech (offline default)
pyttsx3>

# Optional cloud TTS (Edge voices). Friday falls back gracefully if missing.
edge-tts

# Speech recognition (online, free, no API key needed for default engine).
SpeechRe

# Microphone capture (use Python 3.12 on Windows - prebuilt wheels on PyPI)
PyAudio>



# Wake-word detection (CPU friendly, Apache-2.0). Optional - Friday detects
# the wa


# Local LLM-free intent parsing helpers
rapidfuz

# Packaging
pyinstal
```

-- end of requirements.txt (33 lines) --

FILE: run.bat

```
@echo off

setlocal

if not exist .venv (



call .venv\Scripts\activate.bat

endlocal
```

-- end of run.bat (16 lines) --

FILE: scripts\generate_code_pdf.py

```
#!/usr/bin/env python3
"""Gener

from __future__ import annotations

import sys
from dat

ROOT = Path(__file__).resolve().parents[1]
OUTPUT =

# Extensions and explicit paths to include.
INCLUDE_

def _sanitize(text: str) -> str:
    replacem

def collect_files() -> list[Path]:
    files: 
```

```
def _write_line(pdf, line: str, h: float = 4.5, font: str = "Courier", style: str = "", size: int = 7) -> None:
                                                                                                                pdf.set_
```

```
def generate_code_pdf() -> Path:
                                                                                                                try:
```

```
subprocess.check_call([sys.executable, "-m", "pip", "install", "fpdf2", "-q"])
                                                                                                                from fpd
files = collect_files()
                                                                                                                pdf = FP
```

```
# Title page
                                                                                                                usable =
pdf.add_page()
                                                                                                                pdf.set_
```

```
# Source files
                                                                                                                for fpat
```

```
pdf.add_page()
                                                                                                                usable =
```

```
for line in content.splitlines():
_write_
pdf.ln(2)
pdf.set_

OUTPUT.parent.mkdir(parents=True, exist_ok=True)
pdf.outp

def main() -> int:
out = ge

if __name__ == "__main__":
raise Sy
```

-- end of scripts\generate_code_pdf.py (156 lines) --

FILE: scripts\generate_pdfs.py

```
#!/usr/bin/env python3
"""Gener

from __future__ import annotations

import re
import s

ROOT = Path(__file__).resolve().parents[1]
DOCS_MD

def _sanitize(text: str) -> str:
"""Repla

def _write_line(pdf, line: str, h: float = 6, font: str = "Helvetica", style: str = "", size: int = 11) -> None:
pdf.set_

def markdown_to_pdf(md_path: Path, pdf_path: Path) -> None:
from fpd

text = md_path.read_text(encoding="utf-8")
text = _

pdf = FPDF()
pdf.set_

in_code = False
for line
```

```
if line.startswith("# "):
_write_

pdf_path.parent.mkdir(parents=True, exist_ok=True)
pdf.outp

def main() -> int:
try:

subprocess.check_call([sys.executable, "-m", "pip", "install", "fpdf2", "-q"])

DOCS_PDF.mkdir(parents=True, exist_ok=True)

mapping = {
"01_READ

for md_name, pdf_name in mapping.items():
md_path

print(f"\nDone. PDF files saved to:\n {DOCS_PDF}")
return 0

if __name__ == "__main__":
raise Sy
```

-- end of scripts\generate_pdfs.py (129 lines) --

FILE: settings.example.json

```
{
                                                                                "voice":
```

-- end of settings.example.json (65 lines) --

FILE: Start Friday Listener.bat

```
@echo off
REM Alwa
```

-- end of Start Friday Listener.bat (12 lines) --

FILE: test.bat

```
@echo off

setlocal

if not exist .venv (



call .venv\Scripts\activate.bat

endlocal
```

-- end of test.bat (16 lines) --

FILE: tests__init__.py

-- end of tests__init__.py (2 lines) --

FILE: tests\test_intents.py

```
"""Intent classifier tests."""

import unittest

from friday.core.intents import IntentName, classify

class IntentTests(unittest.TestCase):
def asse

def test_open_app(self):
self.ass

def test_open_site(self):
self.ass

def test_weather(self):
self.ass

def test_news(self):
intent =

def test_time(self):
self.ass

def test_joke(self):
self.ass

def test_help(self):
self.ass

def test_search(self):
self.ass

def test_exit(self):
self.ass

def test_greet(self):
self.ass

if __name__ == "__main__":
unittest
```

-- end of tests\test_intents.py (58 lines) --

FILE: tests\test_policy.py

```
"""Safety policy tests - Friday must NEVER allow file access."""

import unittest

from friday.policy import PolicyVerdict, evaluate


class PolicyTests(unittest.TestCase):
# ------

# ------------------------------------------------------------------
# These

def test_blocked_commands_are_denied(self):
for cmd

def test_allowed_commands_pass_policy(self):
```

```
                                                                                                                for cmd

def test_empty_input_is_denied(self):
                                                                                                                self.ass

if __name__ == "__main__":
                                                                                                                unittest
```

-- end of tests\test_policy.py (79 lines) --

FILE: tests\test_router.py

```
"""Router-level integration tests.

These verify that:
                                                                                * Blocke




import unittest
                                                                                from uni

from friday.config import AppSettings
                                                                                from fri


class RouterTests(unittest.TestCase):
                                                                                def setU


def test_blocked_commands_do_not_reach_integrations(self):
                                                                                # If any









def test_greet_returns_text(self):
                                                                                result =


def test_help_returns_text(self):
                                                                                result =


def test_time_intent(self):
                                                                                result =


def test_joke_intent(self):
                                                                                result =


def test_exit_intent_signals_request_exit(self):
                                                                                result =

def test_unknown_command_handled_gracefully(self):
                                                                                result =



if __name__ == "__main__":
                                                                                unittest
```

-- end of tests\test_router.py (65 lines) --